ROUGEHALLA / SYSTEM ARCHITECTURE

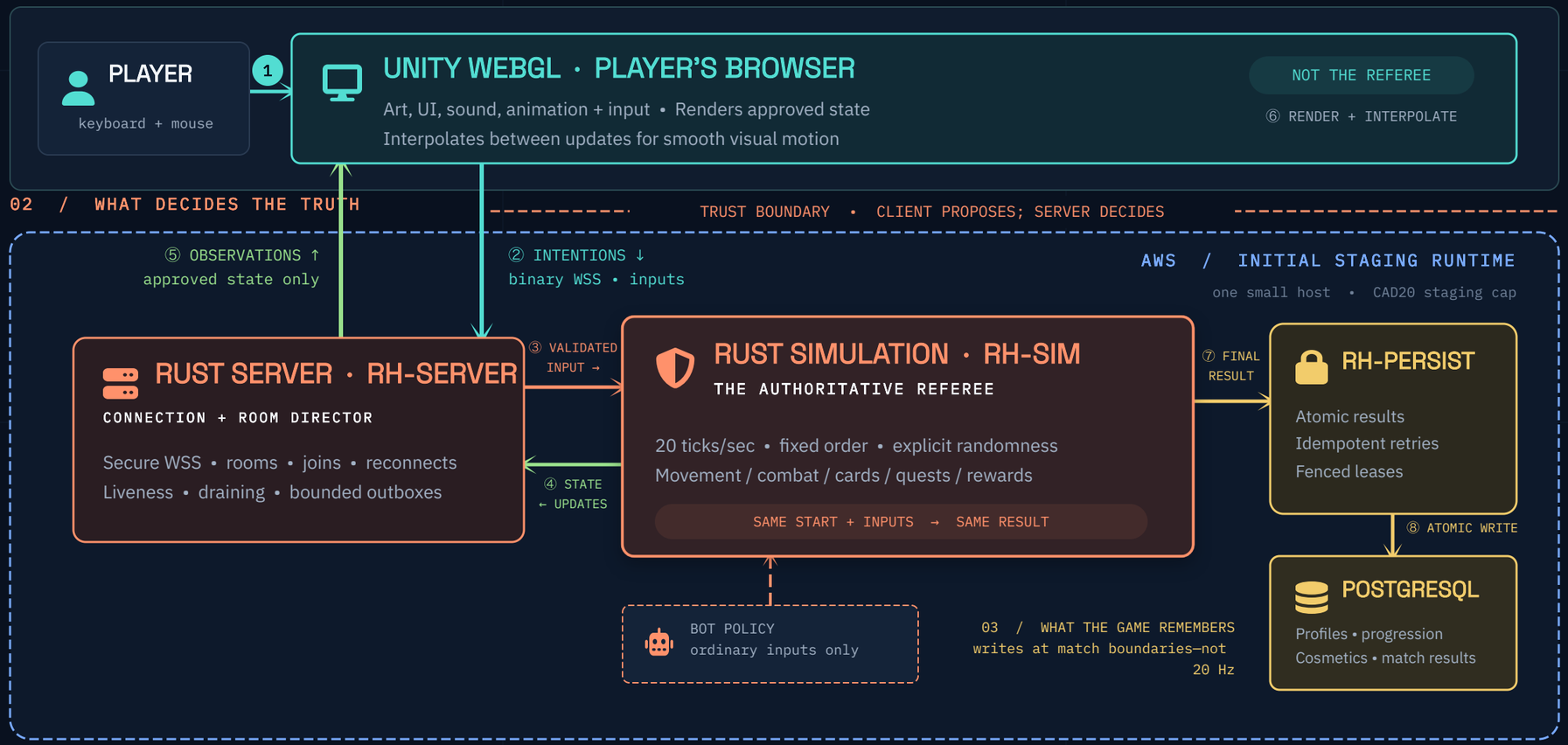
One game. Four responsibilities.

Unity presents. Rust decides. AWS hosts. PostgreSQL remembers.

TARGET ARCHITECTURE

A clear destination—not a claim that the port or production rollout is finished.

01 / WHAT THE PLAYER EXPERIENCES



04 / HOW WE KEEP IT HONEST

evidence + human approval

- **HEADLESS TESTS** — Same simulation, no browser
- **REPLAYS + HASHES** — Same inputs → same result
- **PARITY + BALANCE** — Legacy evidence, declared bands
- **GATES + HUMAN MERGE** — Compile, lint, test, review

TARGET STATE: playable core demonstrated; full mechanics, bots, Unity UI, hosting and production cutover remain incomplete. Weak-hardware performance and real hosting capacity remain release gates. Green tests are not permission to deploy.

WHY THIS SHAPE

- **Unity presents** — Richer visuals and artist workflow without giving the client control over outcomes.
- **Rust decides** — One deterministic authority makes matches reproducible and testable without graphics.
- **AWS hosts** — A small, measured start. Cloud hosting is a venue—not automatic scale or reliability.
- **PostgreSQL remembers** — Permanent rewards are banked safely, without making the database the live game world.

WHY IT CAN BE EFFICIENT

20 Hz SIMULATION ≠ 20 FPS VISUALS

Rust advances a compact world state; Unity renders more often and interpolates between approved updates.

LESS WORK PER TEST MATCH

Headless Rust needs no browser or graphics. Reusable tick memory reduces steady-state allocation.

LESS DATA, FEWER DB WRITES

Binary inputs, player-specific observations and bounded queues limit traffic; PostgreSQL writes at result boundaries.

```
Efficiency is an opportunity, not a guarantee.
Sockets, bandwidth and real load tests determine
match density.
```

START SMALL • MEASURE • SCALE ON EVIDENCE